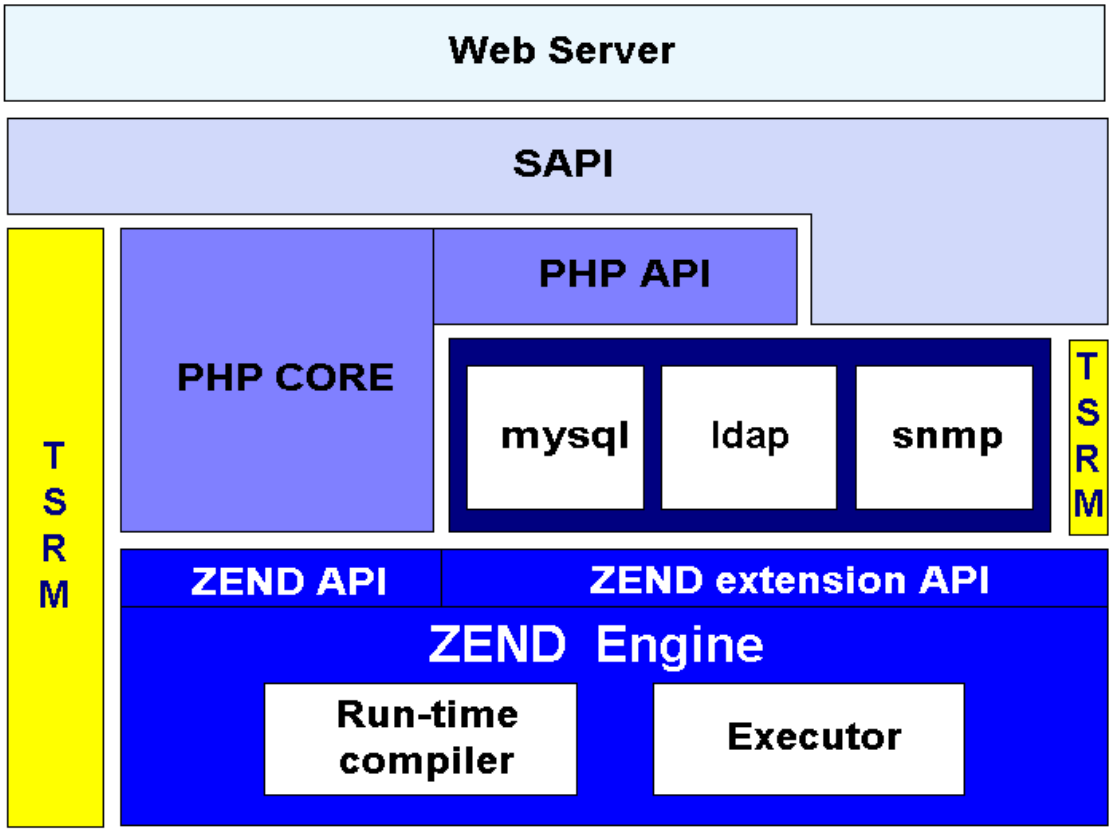
Welcome!

Architecture Diagram



Plan your Extension

Try to plan your extension before you start writing your code. A good way to do this is to write a sample PHP script that illustrates how your extension is going to be accessed from a PHP script.

HTML Tidy Sample Extension

```
<?php
$tidy = tidy_create();
tidy_set_option($tidy, "show-body-only", "yes");
@tidy_parse_string($tidy, "<b>foo <i>bar</b></i>");
tidy_clean_and_repair($tidy);
ob_start();
tidy_output($tidy);
$clean = ob_get_contents();
ob_end_clean();
echo htmlspecialchars($clean);
?>
```

HTML Tidy Sample Extension (OO for PHP4)

```
<?php
$tidy = new tidy();
$tidy->set_option("show-body-only", "yes");
$tidy->parse_string("<b>foo <i>bar</b></i>");
$tidy->clean_and_repair();
$tidy->output();
?>
```

HTML Tidy Sample Extension (OO for PHP5)

```
<?php
$tidy = new tidy();
$tidy->show_body_only = "yes";
$tidy->parse_string("<b>foo <i>bar</b></i>");
$tidy->clean_and_repair();
$tidy->output();
?>
```

Define the functions

From your plan, generate a function definition file

tidy.def

```
resource tidy_create()
    Create a new Tidy document object
void tidy_release(resource tdoc)
    Release an allocated Tidy document object
int tidy_parse_file(resource tdoc, string filename)
    Parse markup from a file
int tidy_parse_string(resource tdoc, string source)
    Parse markup from a string
int tidy_clean_and_repair(resource tdoc)
    Execute configured cleanup and repair operations on parsed markup
string tidy_release_date()
    Get release date (version) for current library
bool tidy_set_option(resource tdoc, string name, string value)
    Get release date (version) for current library
int tidy_status(resource tdoc)
    Get status of current document
int tidy_detected_html_version(resource tdoc)
    Detected HTML version: 0, 2, 3 or 4
bool tidy_detected_xhtml(resource tdoc)
    Input is XHTML?
bool tidy_detected_generic_xml(resource tdoc)
    Input is generic XML (not HTML or XHTML)?
int tidy_error_count(resource tdoc)
    Number of Tidy errors encountered.  If > 0, output is suppressed
    unless TidyForceOutput is set.
int tidy_warning_count(resource tdoc)
    Number of Tidy warnings encountered
int tidy_access_warning_count(resource tdoc)
    Number of Tidy accessibility warnings encountered
int tidy_config_error_count(resource tdoc)
    Number of Tidy configuration errors encountered
int tidy_load_config(resource tdoc, string file [, string encoding])
    Load a Tidy configuration file
bool tidy_file_exists(string file)
    Whether a file exists (with tilde expansion)
int tidy_set_character_encoding(resource tdoc, string encoding)
    Set the input/output character encoding for parsing markup
int tidy_run_diagnostics(resource tdoc)
    Run configured diagnostics on parsed and repaired markup
    (must call tidy_clean_and_repair() first)
int tidy_output(resource tdoc [, string filename])
    Output currently parsed document (to standard output or specified filename)
void tidy_error_summary(resource tdoc)
    Write more complete information about errors
void tidy_general_info(resource tdoc)
    Write more general information about markup
```

Creating a skeleton extension

Run the ext_skel script which creates all the required skeleton files.

```
% cd php4/ext
% ./ext_skel --extname=tidy --proto=tidy.def
Creating directory tidy
Creating basic files: config.m4 Makefile.in .cvsignore tidy.c
                      php_tidy.h CREDITS EXPERIMENTAL
                      tests/001.phpt tidy.php [done].
% ls tidy
config.m4    CREDITS       EXPERIMENTAL    tidy.c
tidy.php     Makefile.in   php_tidy.h      tests/
```

Standalone or Embedded?

At this point you have a choice on how to proceed. You can either develop your extension under the main PHP framework where you run the top-level buildconf script to include your extension in PHP's main configuration script, or you can develop your extension in a completely separate directory where you run the phpize script to populate your extension directory with all the required files.

Embedded

```
% cd tidy
% vi config.m4
% cd ../..
% ./buildconf
% ./configure --[with|enable]-tidy
% make
```

Standalone

```
% mv tidy /tmp
% cd /tmp/tidy
% vi config.m4
% phpize
% ./configure --[with|enable]-tidy
% make
```

Compiling

Compiling the test skeleton extension

```
% cd /tmp/tidy
% vi config.m4
```

The config.m4 file

The config.m4 file adds any extension-specific configuration options. If your extension relies on external libraries and header files, you need to define those in this file and uncomment the --with-tidy section. If your extension does not rely on any external libraries, uncomment the --enable-tidy section.

Configure, build and install

```
% ./configure
% make
% cp modules/tidy.so /usr/local/lib/php
```

Testing your extension

The ext_skel script created a sample tidy.php script. A quick way to check if your extension is working is to edit this script and add dl('tidy.so') to the top. Then run it through your command-line PHP parser:

```
% php -q tidy.php
Congratulations!
You have successfully modified ext/tidy/config.m4.
Module tidy is now compiled into PHP.
```

Structure

The structure of a typical extension contains the following components:

includes

```
#ifdef HAVE_CONFIG_H
#include "config.h"
#endif

#include "php.h"
#include "php_ini.h"
#include "ext/standard/info.h"
#include "php_tidy.h"
```

function_entry

```
function_entry tidy_functions[] = {
    PHP_FE(confirm_tidy_compiled,    NULL)
    {NULL, NULL, NULL}
};
```

module_entry

```
zend_module_entry tidy_module_entry = {
    "tidy",
    tidy_functions,
    PHP_MINIT(tidy),
    PHP_MSHUTDOWN(tidy),
    PHP_RINIT(tidy),
    PHP_RSHUTDOWN(tidy),
    PHP_MINFO(tidy),
    STANDARD_MODULE_PROPERTIES
};
```

PHP_INI_BEGIN

```
PHP_INI_BEGIN()
    STD_PHP_INI_ENTRY("tidy.value",      "42",
                      PHP_INI_ALL, OnUpdateInt, global_value,
                zend_tidy_globals, tidy_globals)
    STD_PHP_INI_ENTRY("tidy.string", "tidybar",
                      PHP_INI_ALL, OnUpdateString, global_string,
                zend_tidy_globals, tidy_globals)
PHP_INI_END()
```

PHP_MINIT_FUNCTION

```
PHP_MINIT_FUNCTION(tidy)
{
    REGISTER_INI_ENTRIES();
    return SUCCESS;
}
```

PHP_MSHUTDOWN_FUNCTION

```
PHP_MSHUTDOWN_FUNCTION(tidy)
{
    UNREGISTER_INI_ENTRIES();
    return SUCCESS;
}
```

PHP_RINIT_FUNCTION

```
PHP_RINIT_FUNCTION(tidy)
{
    return SUCCESS;
}
```

PHP_RSHUTDOWN_FUNCTION

```
PHP_RSHUTDOWN_FUNCTION(tidy)
{
    return SUCCESS;
}
```

PHP_MINFO_FUNCTION

```
PHP_MINFO_FUNCTION(tidy)
{
    php_info_print_table_start();
    php_info_print_table_header(2, "tidy support", "enabled");
    php_info_print_table_end();

    DISPLAY_INI_ENTRIES();
}
```

functions

```
PHP_FUNCTION(confirm_tidy_compiled)
{
    char *arg = NULL;
    int arg_len, len;
    char string[256];

    if (zend_parse_parameters(ZEND_NUM_ARGS() TSRMLS_CC,
                              "s", &arg, &arg_len) == FAILURE) {
        return;
    }

    len = sprintf(string, "Congratulations!
You have successfully modified ext/%.78s/config.m4.
Module %.78s is now compiled into PHP.", "tidy", Z_STRVAL_PP(arg));
    RETURN_STRINGL(string, len, 1);
}
```

the pval/zval datatype

Data in PHP is stored in the pval or zval datatype. Two names for the same thing. They are defined like this:

```
typedef union _zvalue_value {
    long lval;          / long value /
    double dval;        / double value /
    struct {
        char *val;
        int len;
    } str;
    HashTable ht;      / hash table value */
    zend_object obj;
} zvalue_value;

struct _zval_struct {
    zvalue_value value; / value /
    zend_uchar type;    / active type /
    zend_uchar is_ref;
    zend_ushort refcount;
};

typedef struct _zval_struct zval;
typedef zval pval;
```

Convenience Macros

```
#define Z_LVAL(zval)        (zval).value.lval
#define Z_BVAL(zval)        ((zend_bool)(zval).value.lval)
#define Z_DVAL(zval)        (zval).value.dval
#define Z_STRVAL(zval)      (zval).value.str.val
#define Z_STRLEN(zval)      (zval).value.str.len
#define Z_ARRVAL(zval)      (zval).value.ht
#define Z_OBJ(zval)         &(zval).value.obj
#define Z_OBJPROP(zval)     (zval).value.obj.properties
#define Z_OBJCE(zval)       (zval).value.obj.ce
#define Z_RESVAL(zval)      (zval).value.lval

#define Z_LVAL_P(zval_p)        Z_LVAL(*zval_p)
#define Z_BVAL_P(zval_p)        Z_BVAL(*zval_p)
#define Z_DVAL_P(zval_p)        Z_DVAL(*zval_p)
#define Z_STRVAL_P(zval_p)      Z_STRVAL(*zval_p)
#define Z_STRLEN_P(zval_p)      Z_STRLEN(*zval_p)
#define Z_ARRVAL_P(zval_p)      Z_ARRVAL(*zval_p)
#define Z_OBJ_P(zval_p)         Z_OBJ(*zval_p)
#define Z_OBJPROP_P(zval_p)     Z_OBJPROP(*zval_p)
#define Z_OBJCE_P(zval_p)       Z_OBJCE(*zval_p)
#define Z_RESVAL_P(zval_p)      Z_RESVAL(*zval_p)

#define Z_LVAL_PP(zval_pp)      Z_LVAL(**zval_pp)
#define Z_BVAL_PP(zval_pp)      Z_BVAL(**zval_pp)
#define Z_DVAL_PP(zval_pp)      Z_DVAL(**zval_pp)
#define Z_STRVAL_PP(zval_pp)    Z_STRVAL(**zval_pp)
#define Z_STRLEN_PP(zval_pp)    Z_STRLEN(**zval_pp)
#define Z_ARRVAL_PP(zval_pp)    Z_ARRVAL(**zval_pp)
#define Z_OBJ_PP(zval_pp)       Z_OBJ(**zval_pp)
#define Z_OBJPROP_PP(zval_pp)   Z_OBJPROP(**zval_pp)
#define Z_OBJCE_PP(zval_pp)     Z_OBJCE(**zval_pp)
#define Z_RESVAL_PP(zval_pp)    Z_RESVAL(**zval_pp)','100%')
```

Parsing Parameters

In its simplest form you can call ZEND_NUM_ARGS() to check the number of parameters passed to your function.

```
PHP_FUNCTION(count_args)
{
    RETURN_LONG(ZEND_NUM_ARGS());
}
```

You can use the WRONG_PARAM_COUNT macro to output a generic wrong parameter count warning.

```
PHP_FUNCTION(my_func)
{
    if(ZEND_NUM_ARGS() > 1) {
        WRONG_PARAM_COUNT;
    }
    RETURN_TRUE;
}
```

Real Parameter Parsing

To do real parameter handling, use the zend_parse_parameters() function.

```
PHP_FUNCTION(return_arg)
{
    int argc = ZEND_NUM_ARGS();
    long arg;

    if (zend_parse_parameters(argc TSRMLS_CC,
                              "l", &arg) == FAILURE)
        return;

    RETURN_LONG(arg);
}
```

You can make it an optional argument by putting a "|" in the argument format string. Like this:

```
PHP_FUNCTION(return_arg)
{
    int argc = ZEND_NUM_ARGS();
    long arg;

    if (zend_parse_parameters(argc TSRMLS_CC,
                              "|l", &arg) == FAILURE)
        return;

    if(argc) RETURN_LONG(arg)
    else RETURN_FALSE
}
```

Here are all the possible modifiers in the argument format string:

```
'l'  long (integer)
'd'  double (floating point)
's'  string
'b'  boolean
'r'  resource
'a'  array
'o'  any object
'O'  specific object
'z'  mixed
```

So, for example, a function that takes a string, an integer, and array and an optional resource would look like this:

```
PHP_FUNCTION(complex_func)
{
    int argc = ZEND_NUM_ARGS();
    char *str = NULL;
    int str_len;
    int res_id = -1;
    long arg;
    zval *arr = NULL;
    zval *res = NULL;

    if (zend_parse_parameters(argc TSRMLS_CC,
                              "sla|r", &str, &str_len,
                &arg, &arr, &res) == FAILURE)
        return;

    if (res) {
        ZEND_FETCH_RESOURCE(???, ???, res, res_id,
                            "???", ???_rsrc_id);
    }

    RETURN_FALSE;
```

```
}
```

Array Parameters

If your function takes an array parameter. You can check it and access it like this:

```
PHP_FUNCTION(my_arr)
{
    int argc = ZEND_NUM_ARGS();
    zval *tmp, arr = NULL;

    if (zend_parse_parameters(argc TSRMLS_CC,
                              "a", &arr) == FAILURE)
        return;

    zend_hash_internal_pointer_reset(Z_ARRVAL_P(arr));
    while (zend_hash_get_current_data(Z_ARRVAL_P(arr),
                                      (void **)&tmp) == SUCCESS) {
        convert_to_long_ex(tmp);
        php_printf("%ld\n",Z_LVAL_PP(tmp));
        zend_hash_move_forward(Z_ARRVAL_P(arr));
    }
    RETURN_TRUE;
}
```

Here the call to zend_parse_parameters() does the type check. If the user passes a non-array to this function s/he will get a message that looks like this:

```
foo.php(4) : Warning - my_arr() expects argument 1 to be array, integer given
```

If you wanted to write a function that could take either an array of integers or perhaps a single integer and have your function figure it out automatically you do do it like this:

```
PHP_FUNCTION(my_arr)
{
    int argc = ZEND_NUM_ARGS();
    zval *tmp, arg = NULL;

    if (zend_parse_parameters(argc TSRMLS_CC,
                              "z", &arg) == FAILURE)
        return;

    switch(Z_TYPE_P(arg)) {
    case IS_ARRAY:
        zend_hash_internal_pointer_reset(Z_ARRVAL_P(arg));
        while (zend_hash_get_current_data(Z_ARRVAL_P(arg),
                                      (void **)&tmp) == SUCCESS) {
            convert_to_long_ex(tmp);
            php_printf("%ld\n",Z_LVAL_PP(tmp));
            zend_hash_move_forward(Z_ARRVAL_P(arg));
        }
        break;
    case IS_STRING:
    case IS_LONG:
    case IS_DOUBLE:
        convert_to_long_ex(&arg);
        php_printf("%ld\n",Z_LVAL_P(arg));
        break;
    }
    RETURN_TRUE;
}
```

The possible types are:

```
#define IS_NULL     0
#define IS_LONG     1
#define IS_DOUBLE   2
#define IS_STRING   3
```

```
#define IS_ARRAY    4
#define IS_OBJECT   5
#define IS_BOOL     6
#define IS_RESOURCE 7
#define IS_CONSTANT 8
#define IS_CONSTANT_ARRAY   9
```

Returning Values

The simple types are returned using the following convenience macros:

```
#define RETVAL_RESOURCE(l)          ZVAL_RESOURCE(return_value, l)
#define RETVAL_BOOL(b)              ZVAL_BOOL(return_value, b)
#define RETVAL_NULL()               ZVAL_NULL(return_value)
#define RETVAL_LONG(l)              ZVAL_LONG(return_value, l)
#define RETVAL_DOUBLE(d)            ZVAL_DOUBLE(return_value, d)
#define RETVAL_STRING(s, dup)       ZVAL_STRING(return_value, s, dup)
#define RETVAL_STRINGL(s, l, dup)   ZVAL_STRINGL(return_value, s, l, dup)
#define RETVAL_EMPTY_STRING()       ZVAL_EMPTY_STRING(return_value)
#define RETVAL_FALSE                ZVAL_BOOL(return_value, 0)
#define RETVAL_TRUE                 ZVAL_BOOL(return_value, 1)

#define RETURN_RESOURCE(l)          { RETVAL_RESOURCE(l); return; }
#define RETURN_BOOL(b)              { RETVAL_BOOL(b); return; }
#define RETURN_NULL()               { RETVAL_NULL(); return;}
#define RETURN_LONG(l)              { RETVAL_LONG(l); return; }
#define RETURN_DOUBLE(d)            { RETVAL_DOUBLE(d); return; }
#define RETURN_STRING(s, dup)       { RETVAL_STRING(s, dup); return; }
#define RETURN_STRINGL(s, l, dup)   { RETVAL_STRINGL(s, l, dup); return; }
#define RETURN_EMPTY_STRING()       { RETVAL_EMPTY_STRING(); return; }
#define RETURN_FALSE                { RETVAL_FALSE; return; }
#define RETURN_TRUE                 { RETVAL_TRUE; return; }
```

As you can see, returning a value from a function involves setting the special return_value zval to the return value and then just returning from the function call. At the C level, all functions return void.

Simple sum() function:

```
PHP_FUNCTION(my_sum)
{
    int argc = ZEND_NUM_ARGS();
    long arg1;
    long arg2;

    if (zend_parse_parameters(argc TSRMLS_CC,
                              "ll", &arg1, &arg2) == FAILURE)
        return;

    RETURN_LONG(arg1+arg2);
}
```

Here we uppercase every other character of a string.

```
PHP_FUNCTION(my_upper)
{
    char *str = NULL;
    int argc = ZEND_NUM_ARGS();
    int str_len;
    char *p;
    int i;

    if (zend_parse_parameters(argc TSRMLS_CC,
                              "s", &str, &str_len) == FAILURE)
        return;

    for(p=str, i=0; *p; p++,i++) {
        if(i%2) p = toupper(p);
    }
    RETURN_STRINGL(str,str_len,1);
}
```

Returning an Array

Returning an array is a little bit more complex. Here we take a string and a count and return an array with the specified number of elements containing the given string.

```
PHP_FUNCTION(my_array)
{
    char *str = NULL;
    int argc = ZEND_NUM_ARGS();
    int str_len;
    long count;
    int i;

    if (zend_parse_parameters(argc TSRMLS_CC,
                              "sl", &str, &str_len, &count) == FAILURE)
        return;

    array_init(return_value);
    for(i=0; i<count; i++) {
        add_index_stringl(return_value, i, str, str_len, 1);
    }
}
```

So, to return an array you first call array_init() on the return_value zval and then populate the array by calling the various add_* functions. The functions available to populate an array are:

```
add_index_long(zval *arg, uint idx, long n)
add_index_null(zval *arg, uint idx)
add_index_bool(zval *arg, uint idx, int b)
add_index_resource(zval *arg, uint idx, int r)
add_index_double(zval *arg, uint idx, double d)
add_index_string(zval arg, uint idx, char str, int dup)
add_index_stringl(zval arg, uint idx, char str, uint length, int dup)
add_index_zval(zval arg, uint index, zval value)
```

Recall that PHP supports the non-indexed array syntax. eg.

```
<? $a[] = 1; ?>
```

This same concept is available in the extension API using these functions:

```
add_next_index_long(zval *arg, long n)
add_next_index_null(zval *arg)
add_next_index_bool(zval *arg, int b)
add_next_index_resource(zval *arg, int r)
add_next_index_double(zval *arg, double d)
add_next_index_string(zval arg, char str, int dup)
add_next_index_stringl(zval arg, char str, uint length, int dup)
add_next_index_zval(zval arg, zval value)
```

You can also create an array with non-numeric indices. Otherwise known as an associative array using these functions:

```
add_assoc_long(zval arg, char key, long n)
add_assoc_null(zval arg, char key)
add_assoc_bool(zval arg, char key, int b)
add_assoc_resource(zval arg, char key, int r)
add_assoc_double(zval arg, char key, double d)
add_assoc_string(zval arg, char key, char *str, int dup)
add_assoc_stringl(zval arg, char key, char *str, uint length, int dup)
add_assoc_zval(zval arg, char key, zval *value)
```

To return a 2-dimensional array, you simply make new arrays and add them as elements of the top array. Like this:

```
PHP_FUNCTION(my_array)
```

```
{
    char *str = NULL;
    int argc = ZEND_NUM_ARGS();
    int str_len;
    long count;
    int i;
    pval *tmp_arr;

    if (zend_parse_parameters(argc TSRMLS_CC,
                              "sl", &str, &str_len,
                &count) == FAILURE)
        return;

    array_init(return_value);
    for(i=0; i<count; i++) {
        MAKE_STD_ZVAL(tmp_arr);
        array_init(tmp_arr);
        add_assoc_long_ex(tmp_arr,
                      str, str_len,
              count+i);
        add_next_index_zval(return_value, tmp_arr);
    }
}
```

Note the use of add_assoc_long_ex() in the above example. If we know the length of the key, then this is slightly faster than calling add_assoc_long(). The macro definition shows why:

```
int add_assoc_long_ex(zval arg, char key, uint key_len, long n);
#define add_assoc_long(__arg, __key, __n) \\
        add_assoc_long_ex(__arg, __key, strlen(__key)+1, __n)
```

Object Factory Approach

This gets quite a bit more complex than returning simple types, or even arrays. The first thing to do is to declare an extension-wide global class ptr along with the methods available in this object you are creating.

```
static zend_class_entry *my_class_entry_ptr;
static zend_function_entry php_my_class_functions[] = {
    PHP_FE(add,  NULL)
    PHP_FALIAS(del,           my_del,     NULL)
    PHP_FALIAS(list,          my_list,    NULL)
    {NULL, NULL, NULL}
};
```

Next, in our module_init function we initialize and register our class definition.

```
PHP_MINIT_FUNCTION(test) {
    zend_class_entry my_class_entry;

    INIT_CLASS_ENTRY(my_class_entry, "my_class", php_my_class_functions);
    my_class_entry_ptr = zend_register_internal_class(&my_class_entry TSRMLS_CC);
    return SUCCESS;
}
```

And then we create our object factory function which will return an instantiated object.

```
PHP_FUNCTION(my_object) {
{
    char *fn = NULL;
    int argc = ZEND_NUM_ARGS();
    int fn_len;
    long length;
    double price;
    zend_bool setting;

    if (zend_parse_parameters(argc TSRMLS_CC,
                              "sbld", &fn, &fn_len,
                &setting, &length, &price) == FAILURE)
        return;

    object_init_ex(return_value, my_class_entry_ptr);

    add_property_stringl(return_value,
                         "filename", fn, fn_len, 1);
    add_property_bool   (return_value,
                         "toggle", setting ? 0 : 1);
    add_property_long   (return_value,
                         "length", length);
    add_property_double (return_value,
                         "price", price);
}
```

From user space it would be called like this:

```
<?php
  $obj = my_object();
  $obj->add(...);
?>
```

If you want to have the more traditional class instantiation syntax and want to have a constructor run for it. eg.

```
<?php
  $obj = new my_class();
?>
```

Then create a constructor function:

```
PHP_FUNCTION(my_class) {
    char *fn = NULL;
    int argc = ZEND_NUM_ARGS();
    int fn_len;
    long length;
    double price;
    zend_bool setting;

    if (zend_parse_parameters(argc TSRMLS_CC,
                              "sbld", &fn, &fn_len,
                &setting, &length, &price) == FAILURE)
        return;

    add_property_stringl(this_ptr, "filename", fn, fn_len, 1);
    add_property_bool(this_ptr, "toggle", setting?0:1);
    add_property_long(this_ptr, "length", length);
    add_property_double(this_ptr, "price", price);
}
```

Note the use of this_ptr here.

If you want to access properties from within one of the methods, you can do this:

```
pval **tmp;
if(zend_hash_find(HASH_OF(this_ptr),
                  "my_property", 11, (void **)&tmp) == SUCCESS) {
    convert_to_string_ex(tmp);
    php_printf("my_property is set to %s\n", Z_STRVAL_PP(tmp));
}
```

Global Variables

If you need extension-wide global variables, you can't just use standard C-style static variables because PHP needs to be thread-safe. To achieve this, globals are placed in a struct and passed around. To add global variables to your extension, first define them in your header file:

```
ZEND_BEGIN_MODULE_GLOBALS(test)
    int   some_integer;
    char *some_string;
ZEND_END_MODULE_GLOBALS(test)

/ shortcut macros /
#ifdef ZTS
# define TEST_G(v) TSRMG(test_globals_id, zend_test_globals *, v)
#else
# define TEST_G(v) (test_globals.v)
#endif
```

Then in your .c file:

```
ZEND_DECLARE_MODULE_GLOBALS(foo)

static void php_foo_init_globals(zend_foo_globals *foo_globals)
{
    foo_globals->some_integer = 0;
    foo_globals->some_string = NULL;
}
```

And in your MINIT function:

```
ZEND_INIT_MODULE_GLOBALS(foo, php_foo_init_globals, NULL);
```

Now, anywhere you need to get/set one of these variables, use TEST_G(some_integer). This will only work if the test_globals struct is available in the function, of course. All the standard PHP functions will automatically have the globals struct available, but if you write your own utility functions you either have to call TSRMLS_FETCH(); at the top of the function, or better yet, pass in the TSRM struct. Like this:

```
test_utility_function(my_arg TSRMLS_CC);
```

And the function is declared using:

```
static void test_utility_function(int my_arg TSRMLS_DC)
```

SAPI Globals

SAPI is the Server abstraction API and you can access server-related global variables using the SG() macro. Include SAPI.h in your .c file to get the SG macro. You can then access the elements of the sapi_globals_struct which looks like this:

```
typedef struct _sapi_globals_struct {
    void *server_context;
    sapi_request_info request_info;
    sapi_headers_struct sapi_headers;
    int read_post_bytes;
    unsigned char headers_sent;
    struct stat global_stat;
    char *default_mimetype;
    char *default_charset;
    HashTable *rfc1867_uploaded_files;
    long post_max_size;
    int options;
} sapi_globals_struct;
```

To access the default MIME type, you would use:

```
SG(default_mimetype)
```

And to access the request_uri you would use:

```
SG(request_info).request_uri
```

Executor Globals

These are run-time globals. The ones you are likely to be interested in are the symbol_table and active_symbol_table globals. For example, to access the user-space $foo variable from the current global context, you would do this:

```
pval **tmp;
if(zend_hash_find(&EG(symbol_table),
                  "foo", 4, (void **)&tmp) == SUCCESS) {
  RETURN_STRINGL(Z_STRVAL_PP(tmp), Z_STRLEN_PP(tmp), 1);
} else {
  RETURN_FALSE;
}
```

Adding php.ini Entries

Generally the extension-wide global variables are actually configuration options set in the php.ini file, or perhaps in the Apache httpd.conf file. Adding such configuration options for your extension is easy. To add a test.my_ini_setting configuration variable, first follow the instructions for creating a global variable from the last slide. Then in your .c file add:

```
PHP_INI_BEGIN()
    STD_PHP_INI_ENTRY("test.some_integer", "0",
                      PHP_INI_ALL, OnUpdateInt,
              some_integer, zend_test_globals, test_globals)
PHP_INI_END()
```

The arguments to STD_PHP_INI_ENTRY here are:

- entry name
- entry value
- change permissions
- pointer to change notification handler
- corresponding global variable
- globals struct type
- globals struct

You can implement your own change notification handlers using PHP_INI_MH(), but here we will just use the default OnUpdateInt which basically just takes the string in the .ini file and does an atoi() on it. The other built-in handlers are OnUpdateReal(), OnUpdateBool(), OnUpdateString(), and OnUpdateStringUnempty().

After adding your STD_PHP_INI_ENTRY block to your .c file, in your MINIT function after the ZEND_INIT_MODULE_GLOBALS() add this:

```
REGISTER_INI_ENTRIES();
```

And likewise, in your MSHUTDOWN function add:

```
UNREGISTER_INI_ENTRIES();
```

And finally, you can optionally add this to your MINFO function:

```
DISPLAY_INI_ENTRIES();
```

Memory Management

PHP has its own memory management functions which adds a safety net in case you forget to free things. It also helps catch overflows and other memory-related issues. The functions are exactly like their typical C counterparts:

```
emalloc
efree
estrdup
estrndup
ecalloc
erealloc
```

Resources

A generic data container. Used to hold file handles, database connections, or anything else that needs to be initialized and then passed around for other functions to act on. To create a resource in your extension, first declare an extension-wide true global. We don't have to worry about thread-safety here. And then if your resource holds a complex datatype of some sort, typedef it as well at the top of your .c file (on in your header file):

```
static int le_test;

typedef struct _test_le_struct {
    char *name;
    long age;
} test_le_struct;
```

Next we need to create a resource destructor function.

```
static void _php_free_test(zend_rsrc_list_entry *rsrc TSRMLS_DC) {
    test_le_struct test_struct = (test_le_struct )rsrc->ptr;

    efree(test_struct->name);
    efree(test_struct);
}
```

Then in our MINIT function we declare/initialize our resource:

```
le_test = zend_register_list_destructors_ex(_php_free_test,
                                            NULL, "test", module_number);
```

Then we might have a function that looks like this that returns a resource:

```
PHP_FUNCTION(my_init) {
    char *name = NULL;
    int name_len, age;
    test_le_struct *test_struct;

    if (zend_parse_parameters(ZEND_NUM_ARGS() TSRMLS_CC,
                              "sl", &name, &name_len, &age) == FAILURE) {
        return;
    }
    test_struct = emalloc(sizeof(test_le_struct));
    test_struct->name = estrndup(name, name_len);
    test_struct->age = age;
    ZEND_REGISTER_RESOURCE(return_value, test_struct, le_test);
}
```

Followed by other functions that then take a resource as an argument and do something with it:

```
PHP_FUNCTION(my_get)
{
    test_le_struct *test_struct;
    pval *res;

    if (zend_parse_parameters(ZEND_NUM_ARGS() TSRMLS_CC,
                              "r", &res) == FAILURE) {
        return;
    }

    ZEND_FETCH_RESOURCE(test_struct, test_le_struct *,
                        &res, -1, "test", le_test);

    if(!test_struct) RETURN_FALSE;

    array_init(return_value);
    add_assoc_string(return_value, "name", test_struct->name, 1);
```

```
    add_assoc_long(return_value, "age", test_struct->age);
}
```

The End

Index

Architecture Diagram 2
Plan your Extension 3
Define the functions 4
Creating a skeleton extension 5
Compiling 6
Structure 7
the pval/zval datatype 9
Convenience Macros 10
Parsing Parameters 11
Real Parameter Parsing 12
Array Parameters 14
Returning Values 16
Returning an Array 17
Object Factory Approach 19
Global Variables 21
SAPI Globals 22
Adding php.ini Entries 23
Memory Management 24
Resources 25

This presenation has ended.
Click anywhere to return.

_

